Layout-Aware Hierarchical Knowledge Representation for Retrieval-Augmented Generation

Minor Project (IT897) Report

Submitted in partial fulfillment of the requirements for the degree of

MASTER OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

by

KSHITIJ KHEDE

(252IT010)

under the guidance of

Prof. ANANTHANARAYANA V. S.



DEPARTMENT OF INFORMATION TECHNOLOGY

NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA

SURATHKAL, MANGALURU – 575025

September 2026

DECLARATION

I hereby declare that the report of the P.G. Minor Project (IT897) work entitled **"Layout-Aware Hierarchical Knowledge Representation for Retrieval-Augmented Generation"**, submitted to the National Institute of Technology Karnataka, Surathkal, in partial fulfillment of the requirements for the award of the degree of **Master of Technology in Information Technology**, is a bona fide record of the work carried out by me. The material in this report has not been submitted to any university or institution for the award of any degree.

Kshitij Khede (252IT010)
Dept. of Information Technology
NITK Surathkal, Mangaluru

Place: NITK, Surathkal
Date: September 2026

CERTIFICATE

This is to certify that the P.G. Minor Project (IT897) work entitled **"Layout-Aware Hierarchical Knowledge Representation for Retrieval-Augmented Generation"**, submitted by **Kshitij Khede (252IT010)** as the record of the work carried out by him, is accepted as the P.G. Minor Project (IT897) work report submission in partial fulfillment of the requirements for the award of the degree of **Master of Technology in Information Technology** in the Department of Information Technology, NITK Surathkal, India. Duration of this minor project (IT897) is from JUNE 01, 2026 to AUGUST 15, 2026.

Prof. ANANTHANARAYANA V. S.

(Signature of the Guide with Date and Seal)

ACKNOWLEDGEMENT

I am thankful to my guide, **Prof. Ananthanarayana V. S.**, for his guidance and feedback through this minor project. His questions on how the parser handles real document layouts shaped much of the direction of this work.

I also thank the Department of Information Technology, NITK Surathkal, for the facilities and academic environment provided for this project.

Kshitij Khede

ABSTRACT

Retrieval-Augmented Generation (RAG) lets a language model answer from external documents instead of relying only on what it memorised during training. Most RAG pipelines first split a document into fixed-size text chunks and store each chunk as a vector. This step throws away useful structure: the page layout, the order in which text should be read across columns, and the nesting of sections under headings.

This project studies how to keep that structure while turning a PDF into a knowledge representation. The system, SHIA-RAG, works in two stages. First, it recovers the correct reading order of a multi-column page using a geometric rule: text blocks are split at the column gutter and sorted column by column, with a vertical tolerance ($\tau_y = 20$ pt) that groups blocks on the same line. Second, it builds a hierarchy of concept nodes from the document's headings, where each section is attached to its parent section. Section numbering (for example, "3.1" under "3") is the primary signal for attachment; for unnumbered headings the system falls back to a Parent Ranking Engine (PRE) that combines reading-order proximity, TF–IDF text similarity, and heading level. Every hierarchy edge is checked by an upward depth-first search so that the result is acyclic, single-parent, and within a bounded depth.

The system was evaluated on a corpus of eleven research-paper PDFs. Column-aware sorting reduced the mean reading-order inversion rate from 21.8% (naive top-to-bottom extraction) to 0.5%, i.e. reading-order monotonicity rose from 78.2% to 99.5%. Across 196 numbered sections in the corpus, numbering-based attachment placed 100% of sections under the correct parent, while the semantic PRE alone reached 51.5% — confirming that explicit numbering is the stronger signal and that PRE is best used only where numbering is absent. The induced hierarchies contained no cycles and no orphan nodes, and depth stayed within the bound of eight on every document.

Keywords: Retrieval-Augmented Generation; Layout-Aware Parsing; Reading-Order Recovery; Hierarchical Knowledge Representation; Acyclic Graph Invariants; Parent Ranking.

CONTENTS

LIST OF FIGURES **iv**

LIST OF TABLES **v**

ABBREVIATIONS **vi**

1 Introduction **1**

1.1 Background 1

1.2 The Problem with Flat Chunking 1

1.3 Motivation 1

1.4 Scope and Contributions 2

1.5 Organisation of the Report 2

2 Literature Review **3**

2.1 Retrieval-Augmented Generation 3

2.2 Adaptive Retrieval 4

2.3 Tree-Structured Retrieval 4

2.4 Graph-Based Retrieval 4

2.5 Document Layout Analysis 4

2.6 Outcome of Literature Review 4

2.7 Problem Statement 5

2.8 Objectives 5

2.9 Summary 5

3 Methodology **6**

3.1 Overview 6

3.2 Data Model 6

3.3 Layout-Aware Ingestion 6

3.4 Reading-Order Recovery 7

3.5 Heading Classification and Concept Extraction 7

3.6 Hierarchy Induction 9
3.6.1 Candidate Parent Generation (CPG++) 9
3.6.2 Parent Ranking Engine (PRE) 9
3.7 Structural Validation 10
3.8 Confidence Estimation 10
3.9 Summary 11

4 Work Done **12**
4.1 Implementation 12
4.2 Experimental Setup 12
4.3 Reading-Order Results 13
4.4 Hierarchy Results 14
4.5 Case Study: TreeRAG Paper 15
4.6 Summary 15

5 Conclusions and Future Work **16**
5.1 Conclusions 16
5.2 Limitations 16
5.3 Future Work 16

REFERENCES **17**

LIST OF FIGURES

3.1 Processing pipeline from a PDF to a validated concept hierarchy. 6

3.2 Dual-tier data model. Tier 2 concepts are grounded in Tier 1 blocks by an explicit link to their bounding-box evidence. 7

3.3 Naive reading interleaves the two columns; column-aware sorting reads the left column fully before the right column. 8

3.4 Hierarchy induction: candidate generation, parent ranking, and validation. 9

4.1 Mean reading-order monotonicity across the eleven-document corpus. . 13

4.2 Parent Assignment Accuracy by method. Numbering is the strongest signal; PRE is used as a fallback for unnumbered headings. 14

LIST OF TABLES

2.1 Summary of the literature survey for the most relevant related works. . . 3

4.1 Implementation components. 12

4.2 Reading-order inversion rate (%) per document: naive vs. column-aware. 13

4.3 Parent Assignment Accuracy over 196 numbered sections in the corpus. 14

4.4 Full run on the TreeRAG paper. 15

ABBREVIATIONS

Abbreviation	Full Form
RAG	Retrieval-Augmented Generation
PDF	Portable Document Format
CPG	Candidate Parent Generation
PRE	Parent Ranking Engine
KCE	Knowledge Confidence Estimation
ROM	Reading-Order Monotonicity
PAA	Parent Assignment Accuracy
TF–IDF	Term Frequency–Inverse Document Frequency
DFS	Depth-First Search
DAG	Directed Acyclic Graph
LLM	Large Language Model
NLP	Natural Language Processing
OCR	Optical Character Recognition

CHAPTER 1

INTRODUCTION

1.1 Background

Large Language Models (LLMs) generate fluent text, but they have two practical problems. They can state things that are not true, and their knowledge is fixed at training time. Retrieval-Augmented Generation (RAG) reduces both problems by fetching relevant text from an external document store at query time and giving it to the model as context [1, 2]. The model then answers from real source material instead of from memory alone.

A standard RAG pipeline splits each document into fixed-size chunks, embeds every chunk as a vector, and retrieves the nearest chunks to a query [3, 4]. This is simple and works for short factual questions.

1.2 The Problem with Flat Chunking

Fixed-size chunking treats a document as a flat stream of text. On real documents, especially multi-column research papers, this causes three concrete problems.

1. **Scrambled reading order.** A PDF stores text blocks in no fixed order. When a two-column page is read top-to-bottom, text from the left and right columns is interleaved, so sentences from different columns end up mixed together.

2. **Broken context.** A chunk boundary can fall in the middle of a definition or a paragraph, separating a statement from the text that explains it.

3. **Lost hierarchy.** A document has a clear structure — sections nested under headings — but flat chunks record none of it, so a retriever cannot move from a detail up to the section it belongs to.

1.3 Motivation

Humans use layout to read. Font size marks a heading, a number like “3.2” marks a subsection of Section 3, and columns separate two text flows on one page. If a parser ignores these cues, it loses information that is already present in the document. This

project keeps that information: it records where each block sits on the page and how the sections nest, so the representation reflects the document as written.

1.4 Scope and Contributions

This work covers document ingestion, reading-order recovery, and hierarchy construction for digital, text-based academic PDFs. It does not cover the downstream retrieval or answer-generation steps of a full RAG system. The contributions are:

1. A column-aware 2D reading-order algorithm that partitions blocks at the page gutter and sorts each column with a vertical tolerance, correcting the interleaving produced by naive extraction.

2. A hierarchy-induction method that attaches each heading to its parent using section numbering as the primary signal and a Parent Ranking Engine (structural proximity, TF–IDF similarity, and heading level) as a fallback for unnumbered headings.

3. A set of programmatically enforced structural invariants (acyclicity via upward DFS, single parent, and bounded depth), together with topological confidence propagation over the resulting hierarchy.

4. An evaluation on a corpus of eleven research-paper PDFs that measures reading-order monotonicity and parent-assignment accuracy.

1.5 Organisation of the Report

Chapter 2 reviews related work and states the problem and objectives. Chapter 3 describes the method. Chapter 4 reports the implementation and results. Chapter 5 concludes and lists directions for future work.

CHAPTER 2

LITERATURE REVIEW

This chapter reviews retrieval and document-understanding methods related to this project. Table 2.1 summarises the most relevant works.

Table 2.1: Summary of the literature survey for the most relevant related works.

Work	Category	Main idea and limitation
Lewis et al. [1]	Flat RAG	Couples a generator with dense passage retrieval; treats documents as flat passages with no structure.
Guu et al. [2]	Flat RAG	End-to-end retriever pre-training; flat passage retrieval only.
Borgeaud et al. [3]	Flat RAG	Retrieval over trillions of tokens with fixed 64-token chunks; no document structure.
Asai et al. [5]	Adaptive RAG	Learns when to retrieve and self-checks passages; still operates over flat chunks.
Sarthi et al. [6]	Tree RAG	Builds a tree by clustering and summarising chunks; the tree is statistical, not the document's real heading structure.
Tao et al. [7]	Tree RAG	Chunks by markdown headings; depends on explicit heading markup being present.
Lu et al. [8]	Tree RAG	Aligns chunk boundaries with headings; does not enforce graph structure.
Edge et al. [10]	Graph RAG	Extracts an entity graph with LLM prompting; costly and does not ground nodes to page coordinates.
Huang et al. [11]	Layout model	Uses text, image, and 2D position for document understanding; targets extraction/OCR, not structured knowledge graphs.

2.1 Retrieval-Augmented Generation

Lewis et al. [1] and Guu et al. [2] showed that connecting a language model to a document store improves factual accuracy. Borgeaud et al. [3] scaled retrieval to a very large corpus, and Izacard et al. [4] jointly trained the retriever and generator. These systems retrieve flat passages and do not model document structure.

2.2 Adaptive Retrieval

Asai et al. [5] proposed Self-RAG, which trains a model to decide when retrieval is needed and to judge whether retrieved text is relevant. This makes retrieval more selective but still works on flat chunks.

2.3 Tree-Structured Retrieval

Sarthi et al. [6] (RAPTOR) build a tree by clustering chunks and summarising each cluster; the tree comes from embedding statistics rather than the document's headings. Tao et al. [7] (TreeRAG) chunk by markdown headings, which requires the headings to be explicitly marked up. Lu et al. [8] (HiChunk) align chunk boundaries with headings and report gains over fixed-size chunking. Fatehkia et al. [9] inject a hand-built entity tree into the prompt.

2.4 Graph-Based Retrieval

Edge et al. [10] (GraphRAG) build an entity graph by prompting an LLM on each text unit and then cluster it into community summaries. This captures relations but needs many LLM calls and does not link nodes back to their position on the page.

2.5 Document Layout Analysis

Document layout analysis recovers logical structure from spatial geometry. The document spectrum by O'Gorman [12] and geometric algorithms by Breuel [13] group text blocks by spatial proximity. Vision-language models such as LayoutLMv3 [11] incorporate 2D coordinates into transformer architectures. These techniques focus primarily on information extraction rather than on constructing validated knowledge hierarchies for retrieval.

2.6 Outcome of Literature Review

The reviewed methods fall into two groups. Retrieval methods (flat, adaptive, tree, and graph) focus on how to store and fetch text, and mostly discard page geometry during preprocessing. Layout-analysis methods recover geometry but stop at extraction and do not produce a structured, validated hierarchy. Few methods keep the physical reading order of multi-column pages and, at the same time, enforce explicit structural guarantees on the resulting hierarchy.

2.7 Problem Statement

To convert a multi-column PDF into a hierarchically structured knowledge representation while preserving its physical reading order and enforcing verifiable structural constraints.

2.8 Objectives

1. To ingest digital PDF documents and extract text blocks with their 2D bounding boxes and font attributes.
2. To recover the correct reading order of multi-column pages using a geometric, column-aware sorting algorithm.
3. To induce a hierarchy of concept nodes from document headings, using section numbering as the primary signal and a Parent Ranking Engine as a fallback.
4. To enforce structural invariants (acyclicity, single parent, bounded depth) and propagate confidence over the hierarchy.
5. To evaluate reading-order recovery and parent-assignment accuracy on a corpus of research-paper PDFs.

2.9 Summary

This chapter reviewed flat, adaptive, tree, and graph retrieval methods and layout analysis, noting that reading order and structural guarantees are usually not addressed together. It then stated the problem and the objectives for this project.

CHAPTER 3

METHODOLOGY

3.1 Overview

The system takes a PDF and produces a hierarchy of concept nodes. Figure 3.1 shows the stages: ingestion, reading-order recovery, heading classification, concept extraction, hierarchy induction, validation, and confidence estimation.

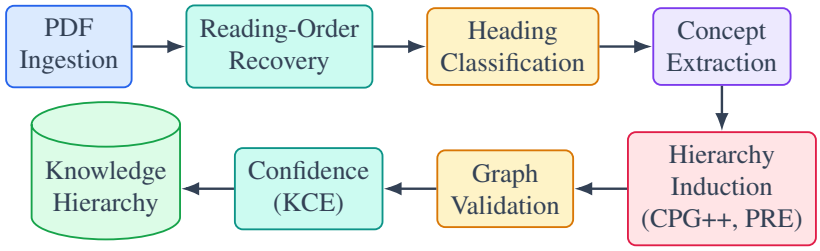


Figure 3.1: Processing pipeline from a PDF to a validated concept hierarchy.

3.2 Data Model

The representation has two tiers, kept separate so that physical layout is not mixed with semantic content. Figure 3.2 illustrates the two tiers and the link between them.

Tier 1 stores physical layout: each **TextBlock** holds its text, page number, font size, font weight, reading-order rank, and bounding box (x_0, y_0, x_1, y_1) in page points. Tier 2 stores semantics: each **ConceptNode** holds a canonical name, an optional definition, a heading level, a confidence score, and a link to the Tier 1 block it came from. A **HierarchyEdge** connects a child concept to its single parent. The hierarchy is a set of rooted trees (a forest); every non-root node has exactly one parent.

3.3 Layout-Aware Ingestion

The ingestion step opens the PDF and reads each page's text with position and font information. For each line it records the bounding box, the most common font size, and whether the font is bold. The reference implementation uses pdfplumber; any extractor that reports per-block coordinates and font size (for example, PyMuPDF) can be used in its place. Scanned or image-only pages would first need OCR, which is outside the scope of this work.

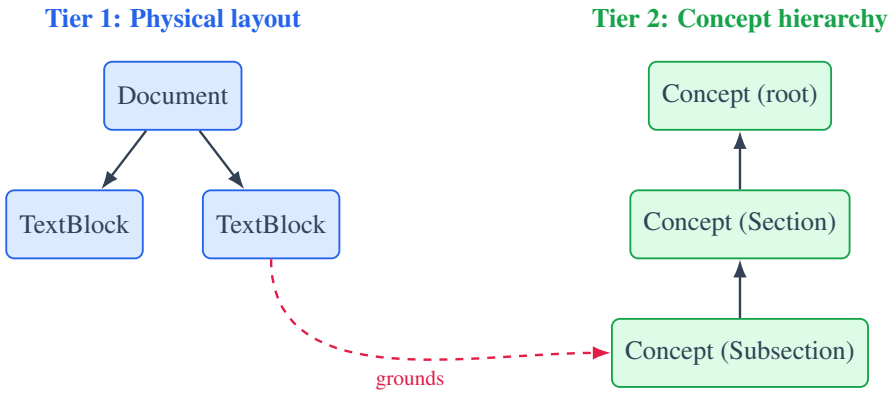


Figure 3.2: Dual-tier data model. Tier 2 concepts are grounded in Tier 1 blocks by an explicit link to their bounding-box evidence.

3.4 Reading-Order Recovery

A PDF does not store text in reading order, and reading a two-column page top-to-bottom mixes both columns [11]. Guided by geometric layout-analysis principles [12, 13], the system recovers reading order by splitting the page at the gutter and sorting each column separately.

Let a page have width W and height H, and let each block b have a bounding box (x_0, y_0, x_1, y_1). The algorithm proceeds in three steps.

1. **Spanning blocks.** A block that is wider than $0.60W$ spans both columns. If it also sits near the top ($y_0 < 0.30H$) it is a title or full-width header and is placed before the columns; otherwise it is placed after them.

2. **Column split.** Remaining blocks go to the left column if $x_1 \leq W/2$ and to the right column otherwise.

3. **Within-column order.** Each column is sorted by top coordinate y_0. Blocks whose tops differ by at most $\tau_y = 20$ pt are treated as one horizontal line and ordered left to right by x_0.

The final order is: top-spanning blocks, then the whole left column, then the whole right column, then bottom-spanning blocks. Algorithm 1 states the procedure, and Figure 3.3 contrasts it with naive top-to-bottom reading.

3.5 Heading Classification and Concept Extraction

The system estimates the body font size as the most common size among long text lines. A block is treated as a heading if it starts with a section number (matching

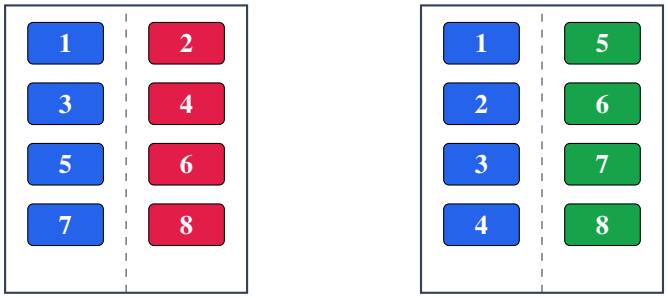


Figure 3.3: Naive reading interleaves the two columns; column-aware sorting reads the left column fully before the right column.

Algorithm 1 Column-Aware 2D Reading-Order Sorting

Require: Blocks $\mathscr{B}$ of a page, width W, height H, tolerance $\tau_y = 20$ pt
Ensure: Ordered blocks $\mathscr{B}_{\text{sorted}}$

$S_{\text{top}}, S_{\text{bot}}, C_1, C_2 \leftarrow [], [], [], []$
for all $b \in \mathscr{B}$ **do**
  if $(b.x_1 - b.x_0) > 0.60\,W$ **and** $b.y_0 < 0.30\,H$ **then**
    append b to S_{top}
  else if $(b.x_1 - b.x_0) > 0.60\,W$ **then**
    append b to S_{bot}
  else if $b.x_1 \leq W/2$ **then**
    append b to C_1
  else
    append b to C_2
  end if
end for
for all column $C \in \{C_1, C_2\}$ **do**
  sort C by y_0; group blocks with $|y_{0,i} - y_{0,j}| \leq \tau_y$ into a line; order each line by x_0
end for
sort $S_{\text{top}}, S_{\text{bot}}$ by y_0
return $S_{\text{top}} \oplus C_1 \oplus C_2 \oplus S_{\text{bot}}$

`^\d+(\.\d+)*`) or if its font is at least 1.25× the body size. For a numbered heading, the level is the count of number components: "3" is level 1, "3.2" is level 2, "3.2.1" is level 3. For an unnumbered heading, the level is set from its relative font size. Each heading becomes a concept node whose name is the heading text and whose evidence link points to the source block's bounding box. If the first body line after a heading begins with a definitional cue ("is", "are", "refers to", "is defined as", "denotes"), it is stored as the concept's definition.

3.6 Hierarchy Induction

Each concept must be attached to a parent. Figure 3.4 shows the two-step procedure: Candidate Parent Generation (CPG++) narrows the candidates, and the Parent Ranking Engine (PRE) picks one. Both are formulated in this work.

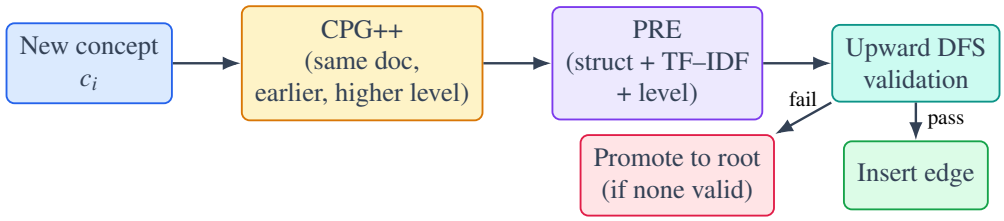


Figure 3.4: Hierarchy induction: candidate generation, parent ranking, and validation.

3.6.1 Candidate Parent Generation (CPG++)

For a concept c_i, the candidate set contains concepts from the same document that appear earlier in reading order and have a higher structural level (a smaller level index):

$$\mathrm{CPG}(c_i) = \{u : \mathrm{doc}(u) = \mathrm{doc}(c_i) \ \wedge \ \mathrm{order}(u) < \mathrm{order}(c_i) \ \wedge \ \mathrm{level}(u) < \mathrm{level}(c_i)\}. \tag{3.1}$$

If this set is empty (for example, when heading levels are ambiguous), the constraint on level is dropped and any earlier concept in the same document is a candidate. If there is still no candidate, c_i becomes a root.

3.6.2 Parent Ranking Engine (PRE)

When several candidates exist, PRE scores each candidate u and picks the best:

$$\mathrm{Score}(u, c_i) = w_{\mathrm{struct}} S_{\mathrm{struct}} + w_{\mathrm{sem}} S_{\mathrm{sem}} + w_{\mathrm{level}} S_{\mathrm{level}}, \tag{3.2}$$

with weights $w_{\mathrm{struct}} = 0.40$, $w_{\mathrm{sem}} = 0.40$, and $w_{\mathrm{level}} = 0.20$. The three terms are each bounded in $[0, 1]$:

- $S_{\text{struct}} = 1/(1+\log(1+\Delta))$, where Δ is the reading-order gap between u and c_i; closer candidates score higher.
- S_{sem} is the cosine similarity between the TF–IDF vectors [14] of the two concepts' text; more similar text scores higher.
- $S_{\text{level}} = \max(0,\, 1-0.2(\Delta\text{level}-1))$, which favours an immediate parent ($\Delta\text{level} = 1$) over a distant ancestor.

A dense sentence encoder can replace the TF–IDF term without changing the rest of the method; TF–IDF is used here because it needs no external model and is fully reproducible. As the results in Chapter 4 show, section numbering is a stronger signal than S_{sem} for numbered documents, so PRE is used mainly for unnumbered headings.

3.7 Structural Validation

Before an edge (u, c_i) is inserted, it is checked against three invariants:

- **Acyclicity.** An upward DFS starts at u and follows parent links. If it reaches c_i, the edge would create a cycle and is rejected. Because a parent always precedes its child in reading order, and the traversal is bounded by the maximum depth, this check is inexpensive.
- **Single parent.** A non-root concept keeps exactly one parent.
- **Bounded depth.** No node may exceed depth $D_{\max} = 8$.

If the chosen edge fails validation, the concept is promoted to a root rather than left unattached, so there are no orphan nodes. Algorithm 2 states the check.

3.8 Confidence Estimation

Each concept gets a confidence score that reflects how clearly it was detected and how reliable its ancestors are. A local score starts at 0.5 and adds 0.15 if the font is larger than body text, 0.10 if bold, 0.15 if it has a section number, and 0.10 if a definition was found, capped at 1.0. Scores are then propagated from roots to leaves in topological order using Kahn's algorithm [15]: a child's score is $C(v) = C_{\text{local}}(v)\,(\delta + (1-\delta)C(\text{parent}))$ with damping $\delta = 0.85$. Processing parents before children keeps the pass deterministic.

Algorithm 2 Upward DFS Cycle and Depth Check

Require: candidate edge (u, v) (parent u, child v), parent map
Ensure: True if the edge is valid

if $u = v$ **then**
  return False
end if
if $\text{depth}(u) + 1 > D_{\max}$ **then**
  return False
end if
$stack \leftarrow [u]$, $seen \leftarrow \{v\}$
while $stack$ not empty **do**
  $x \leftarrow \text{pop}(stack)$
  if $x = v$ **then**
    return False {cycle}
  end if
  if $x \notin seen$ **then**
    add x to $seen$; push parent(x) if it exists
  end if
end while
return True

3.9 Summary

This chapter described the method: column-aware reading-order recovery, heading-based concept extraction, hierarchy induction with CPG++ and PRE, upward-DFS validation, and topological confidence propagation.

CHAPTER 4

WORK DONE

4.1 Implementation

The system is implemented in Python. Table 4.1 lists the main components. PDF text and coordinates are read with pdfplumber; TF–IDF similarity uses scikit-learn; numerical work uses NumPy; and data objects are validated with Pydantic. The code is organised as six modules that follow the pipeline in Figure 3.1: ingestion, structural parsing (reading order and heading classification), concept extraction, hierarchy building (CPG++ and PRE), graph validation, and confidence scoring.

Table 4.1: Implementation components.

Component	Role
Python 3	Implementation language
pdfplumber	Extract text blocks with bounding boxes and font size
scikit-learn	TF–IDF vectors and cosine similarity for the PRE semantic term
NumPy	Vector and array operations
Pydantic	Validation of Tier 1 and Tier 2 data objects

4.2 Experimental Setup

The system was evaluated on a corpus of eleven research-paper PDFs in the retrieval-augmented generation area (for example, TreeRAG, GraphRAG, RAPTOR, Self-RAG, HiChunk, RETRO, Atlas, T-RAG, REALM, HAT-RAG, and the original RAG paper). These are digital, mostly two-column academic PDFs, which is the document type this project targets.

Two things were measured. Reading order was measured by the pairwise inversion rate against a column-first reference order defined by page geometry (left column top to bottom, then right column). Hierarchy quality was measured by Parent Assignment Accuracy (PAA): the fraction of numbered sections whose assigned parent matches the parent implied by the section numbering (“3.1” under “3”).

4.3 Reading-Order Results

Table 4.2 compares naive top-to-bottom extraction with column-aware sorting. Across the corpus, naive extraction had a mean inversion rate of 21.8% (reading-order monotonicity of 78.2%), while column-aware sorting reduced it to 0.5% (monotonicity of 99.5%). Figure 4.1 shows the per-strategy means.

Table 4.2: Reading-order inversion rate (%) per document: naive vs. column-aware.

Document	Naive inversion (%)	Column-aware (%)
GraphRAG	19.1	0.1
Atlas	18.9	0.6
HiChunk	23.5	0.3
HAT-RAG	23.6	1.3
RETRO	23.1	0.4
Self-RAG	22.7	0.5
RAPTOR	20.9	0.3
RAG (Lewis)	18.6	0.4
REALM	26.2	0.8
T-RAG	23.2	0.2
TreeRAG	20.4	0.7
Mean	**21.8**	**0.5**

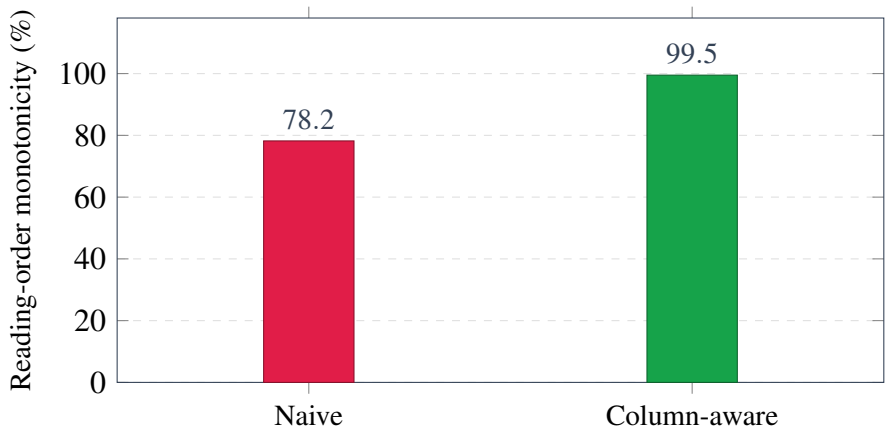


Figure 4.1: Mean reading-order monotonicity across the eleven-document corpus.

The gain comes from separating the columns before sorting. Naive extraction interleaves the columns because blocks at similar heights in different columns are read one after another. The residual 0.5% comes mostly from full-width figures or tables placed in the middle of a page, which the gutter rule places after the columns.

4.4 Hierarchy Results

Table 4.3 reports Parent Assignment Accuracy over the 196 numbered sections in the corpus, for four attachment methods. Section numbering places every numbered section under the correct parent. The semantic Parent Ranking Engine, used on its own, reaches 51.5%: it is better than a sequential or lexical rule but clearly weaker than numbering, because the section number is an almost perfect signal that text similarity cannot improve on. Figure 4.2 shows the comparison.

Table 4.3: Parent Assignment Accuracy over 196 numbered sections in the corpus.

Attachment method	PAA (%)
Sequential (previous heading)	17.9
Lexical prefix overlap	34.7
Parent Ranking Engine (TF–IDF semantic)	51.5
Section numbering (primary signal)	**100.0**

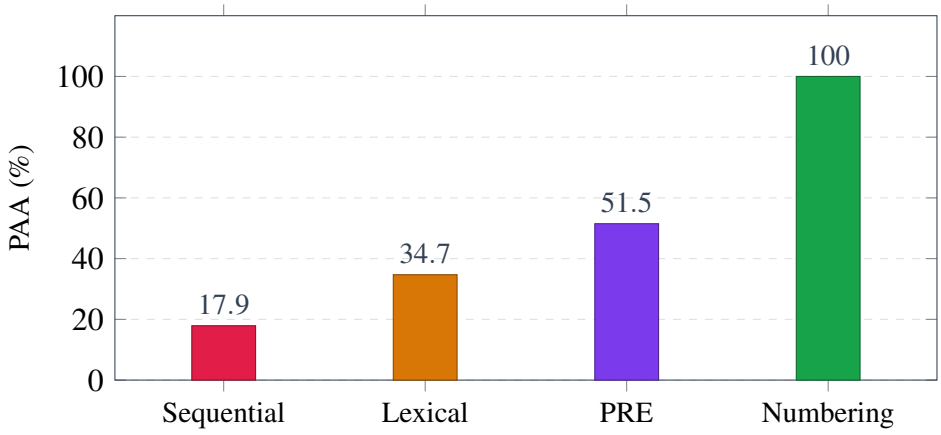


Figure 4.2: Parent Assignment Accuracy by method. Numbering is the strongest signal; PRE is used as a fallback for unnumbered headings.

This result guided the design: the system uses numbering as the primary signal and calls the Parent Ranking Engine only for headings that carry no number (for example, “Abstract” or “References”). The combined policy keeps the near-perfect accuracy on numbered sections while still attaching unnumbered headings sensibly.

Across all eleven documents, the induced hierarchies contained **no cycles** and **no orphan nodes**, and the depth stayed within the bound of eight. These are a direct result of the validation step in Section 3.7: cycles are rejected by the upward DFS, and any concept that cannot be attached is promoted to a root instead of being dropped.

4.5 Case Study: TreeRAG Paper

Table 4.4 summarises a full run on the 16-page, two-column TreeRAG paper. The parser extracted 262 text blocks and 15 concept nodes, of which 13 are the paper's numbered sections. Every numbered section was attached to its correct parent (for example, "3.1 Tree-Chunking" under "3 TreeRAG", and "3.1.1" under "3.1"). The hierarchy had one root, a maximum depth of three, no cycles, and no orphans, and the run took about 1.8 seconds.

Table 4.4: Full run on the TreeRAG paper.

Measurement	Value
Pages	16
Text blocks extracted	262
Concept nodes	15
Numbered sections	13 (all attached correctly)
Hierarchy edges	14
Maximum depth	3
Cycles / orphans	0 / 0
Mean confidence	0.62
Runtime	$\approx$1.8 s

4.6 Summary

On the eleven-document corpus, column-aware sorting raised reading-order monotonicity from 78.2% to 99.5%. Section numbering attached 100% of numbered sections correctly, the Parent Ranking Engine reached 51.5% on its own, and every hierarchy was acyclic, orphan-free, and within the depth bound.

CHAPTER 5

CONCLUSIONS AND FUTURE WORK

5.1 Conclusions

This project developed a layout-aware parser that converts multi-column PDFs into validated concept hierarchies. First, column-aware sorting reduced the mean reading-order inversion rate from 21.8% to 0.5% across eleven research papers. Second, section numbering proved the most reliable hierarchy signal (100% accuracy), while the Parent Ranking Engine served as an effective semantic fallback for unnumbered headings (51.5% standalone). Structural validation programmatically ensured acyclicity, zero orphan nodes, and bounded depth across all evaluated documents.

5.2 Limitations

1. Assumes symmetric two-column layouts split at $W/2$; mid-page full-width floats are not handled ideally.
2. Evaluated on eleven academic PDFs; non-academic documents may behave differently.
3. Requires digital text; scanned pages necessitate optical character recognition.
4. TF–IDF semantic similarity is less effective on heavily paraphrased headings than dense encoders.
5. Ground truth is based on section numbering, which does not assess unnumbered heading semantics.

5.3 Future Work

1. Replace TF–IDF with a dense sentence encoder for unnumbered headings.
2. Automatically detect column count and sequence mid-page full-width floats.
3. Ground generated answers to physical page regions using Tier 1 bounding boxes.
4. Construct a human-annotated hierarchy to evaluate unnumbered attachments directly.
5. Test parser generalisation across technical manuals and corporate reports.

REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[2] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M. Chang. REALM: Retrieval-augmented language model pre-training. In *Proc. 37th Int. Conf. on Machine Learning (ICML)*, pp. 3929–3938, 2020.

[3] S. Borgeaud, A. Mensch, J. Hoffmann, et al. Improving language models by retrieving from trillions of tokens. In *Proc. 39th Int. Conf. on Machine Learning (ICML)*, pp. 2206–2240, 2022.

[4] G. Izacard, P. Lewis, M. Lomeli, et al. Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research*, 24(251):1–43, 2023.

[5] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *Int. Conf. on Learning Representations (ICLR)*, 2024.

[6] P. Sarthi, S. Abdullah, A. Tuli, et al. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *Int. Conf. on Learning Representations (ICLR)*, 2024.

[7] W. Tao, X. Xing, Y. Chen, L. Huang, and X. Xu. TreeRAG: Unleashing the power of hierarchical storage for enhanced knowledge retrieval in long documents. In *Findings of the ACL 2025*, pp. 356–371, 2025.

[8] W. Lu, K. Chen, Z. Shen, R. Qiao, and X. Sun. HiChunk: Evaluating and enhancing retrieval-augmented generation with hierarchical chunking. In *Proc. 64th Annual Meeting of the ACL*, pp. 29738–29753, 2026.

[9] M. Fatehkia, J. Kim Lucas, and S. Chawla. T-RAG: Lessons from the LLM trenches. *arXiv preprint arXiv:2402.07483*, 2024.

[10] D. Edge, H. Trinh, N. Cheng, et al. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[11] Y. Huang, T. Lv, L. Cui, Y. Lu, and F. Wei. LayoutLMv3: Pre-training for document AI with unified text and image masking. In *Proc. 30th ACM Int. Conf. on Multimedia*, pp. 4083–4091, 2022.

[12] L. O'Gorman. The document spectrum for page layout analysis. *IEEE Trans. on Pattern Analysis and Machine Intelligence*, 15(11):1162–1173, 1993.

[13] T. M. Breuel. Two geometric algorithms for layout analysis. In *Document Analysis Systems*, pp. 188–199, Springer, 2002.

[14] G. Salton and M. J. McGill. *Introduction to Modern Information Retrieval*. McGraw-Hill, 1983.

[15] A. B. Kahn. Topological sorting of large networks. *Communications of the ACM*, 5(11):558–562, 1962.

[16] R. Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.

[17] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein. *Introduction to Algorithms*, 3rd ed. MIT Press, 2009.